

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федерального государственного бюджетного образовательного учреждения
высшего образования
**«РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ
Г.В.ПЛЕХАНОВА»**
Техникум Пермского института (филиала)

Практическая работа №2
по дисциплине «Поддержка и тестирование программных модулей»
по теме: Разработка приложения «Калькулятор»

Руководитель

_____ /Д.Б. Берестов /

«____» _____ 2026 г.

Исполнитель

_____ /А.С.Нефедов/

«____» _____ 2026 г.

Пермь, 2026 г.

Оглавление

Оглавление

Введение.....	3
Проектирование структуры приложения.....	3
Реализация библиотеки классов CalculatorLibrary	4
Разработка пользовательского интерфейса (WPF).....	5
Реализация логики работы интерфейса	6
Модульное тестирование	9
Заключение	10

Введение

В рамках практической работы №2 по дисциплине «Поддержка и тестирование программных модулей» была поставлена задача разработать приложение «Калькулятор» с использованием технологии WPF и Microsoft Visual Studio.

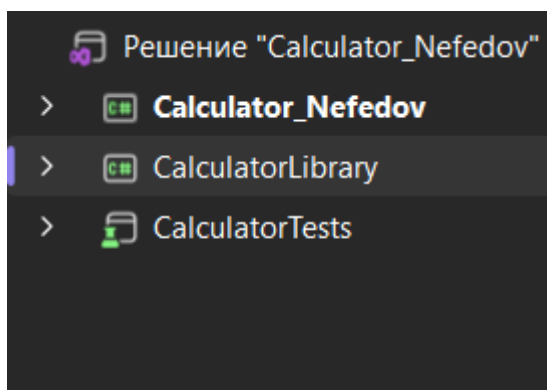
Основная цель работы заключалась в создании программного продукта, соответствующего принципам модульной архитектуры, где логика отделена от пользовательского интерфейса и вынесена в отдельную библиотеку классов.

В работе также требовалось реализовать модульные тесты для проверки корректности работы всех математических операций, реализованных в библиотеке классов.

Проектирование структуры приложения

Приложение имеет трёхуровневую структуру:

1. Библиотека классов (CalculatorLibrary) — содержит класс Calculator, в котором реализованы методы для выполнения математических операций.
2. WPF-приложение (Calculator_Nefedov) — отвечает за отображение пользовательского интерфейса и обработку событий (нажатие кнопок, отображение результата).
3. Модульные тесты (CalculatorTests) — содержат набор тестов для проверки корректности работы библиотеки.



Реализация библиотеки классов CalculatorLibrary

Библиотека классов представляет собой отдельный проект, содержащий класс Calculator с методами для выполнения арифметических операций.

```
namespace CalculatorLibrary
{
    public class Calculator
    {
        public double Add(double a, double b)
        {
            return a + b;
        }

        public double Subtract(double a, double b)
        {
            return a - b;
        }

        public double Multiply(double a, double b)
        {
            return a * b;
        }

        public double Divide(double a, double b)
        {
            if (b == 0)
            {
                throw new DivideByZeroException("Деление на ноль невозможно");
            }
            return a / b;
        }

        public double Power(double a, double b)
        {
            return Math.Pow(a,b);
        }

        public double Calculate(double a, double b, string Operation)
    }
```

Рисунок 2 – CalculatorLibrary

Метод Calculate является универсальным и вызывает соответствующий метод в зависимости от переданной операции.

При делении на ноль выбрасывается исключение DivideByZeroException.

Для возведения в степень используется встроенная функция Math.Pow.

```

        return a / b;
    }
    Ссылка: 1
    public double Power(double a, double b)
    {
        return Math.Pow(a,b);
    }

    Ссылка: 0
    public double Calculate(double a, double b, string Operation)
    {
        switch (Operation)
        {
            case "+":
                return Add(a,b);
            case "-":
                return Subtract(a, b);
            case "*":
                return Multiply(a, b);
            case "/":
                return Divide(a, b);
            case "^":
                return Power(a, b);
            default:
                throw new InvalidOperationException("Неизвестная операция");
        }
    }
}

```

Рисунок 3 – Метод Calculate

Разработка пользовательского интерфейса (WPF)

Пользовательский интерфейс был создан с использованием технологии WPF (Windows Presentation Foundation).

Интерфейс включает:

1. Поле для отображения результата — TextBox с именем ResultWindow, доступное только для чтения.
2. Кнопки для ввода цифр — от 0 до 9.
3. Кнопки операций — «+», «-», «*», «/», «^».
4. Кнопку вычисления результата — «=».
5. Кнопку очистки — «C».

Каждая кнопка обрабатывает событие Click, которое вызывает соответствующий обработчик в коде.

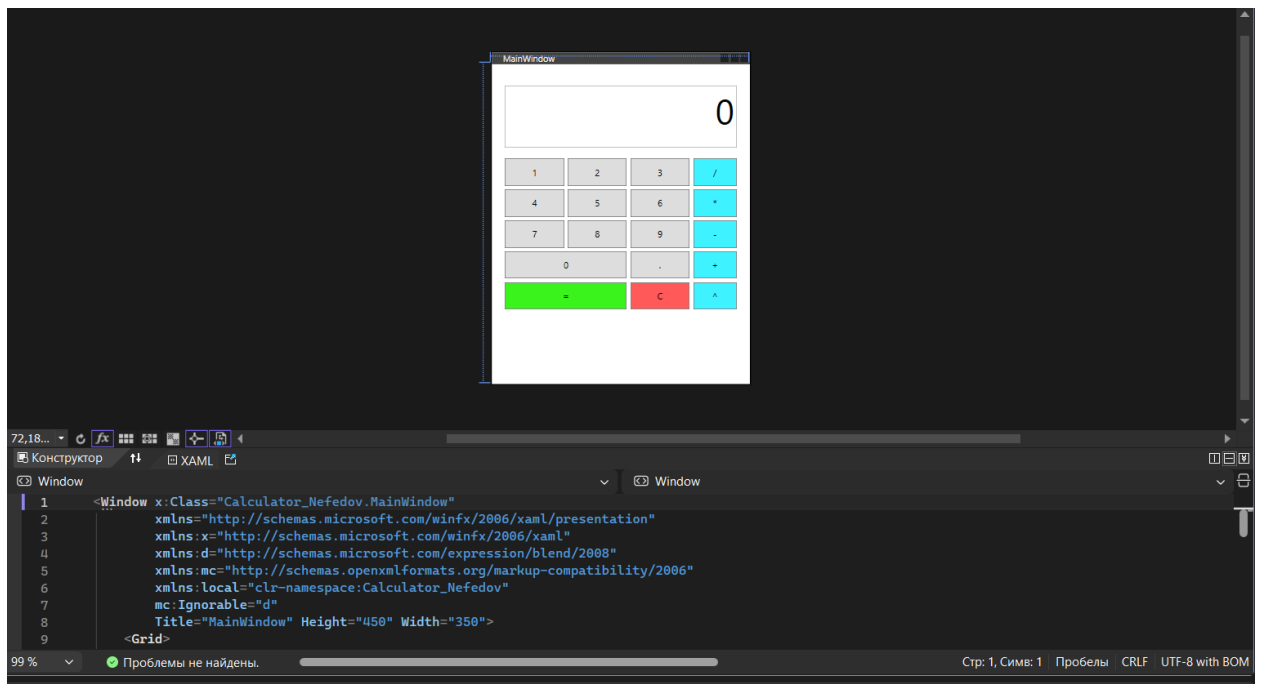


Рисунок 4 – Пользовательский интерфейс

Реализация логики работы интерфейса

В коде файла MainWindow.xaml.cs реализована логика взаимодействия пользователя с калькулятором.

Основные переменные состояния:

1. currentValue – Текущее накопленное значение
2. pendingOperation – Ожидающая операция
3. isNewInput – Флаг для нового ввода после операции

Основные методы:

1. ButtonLogic — добавляет символ к текущему вводу и обновляет отображение.

Ссылка: 11

```
private void ButtonLogic(object sender, RoutedEventArgs e)
{
    Button button = (Button)sender;
    string content = button.Content.ToString();

    if (isNewInput)
    {
        CurInput = "";
        isNewInput = false;
    }

    if (content == "." && CurInput.Contains("."))
        return;

    if (CurInput == "0" && content != ".")
    {
        CurInput = content;
    }
    else
    {
        CurInput += content;
    }

    UpdateDisplay();
}
```

Рисунок 5 – ButtonLogic

2. OperationLogic — обрабатывает выбор операции.

```

private void OperationLogic(string Op)
{
    if (!isNewInput && CurInput != "")
    {
        double currentNumber = double.Parse(CurInput, CultureInfo.InvariantCulture);

        if (currentValue != null && pendingOperation != null)
        {
            currentValue = Calc.Calculate((double)currentValue, currentNumber, pendingOperation);
            CurInput = currentValue.ToString();
            UpdateDisplay();
        }
        else
        {
            currentValue = currentNumber;
        }
    }

    if (Op == "-" && CurInput == "" && currentValue == null)
    {
        CurInput = "-";
        isNewInput = false;
        UpdateDisplay();
        return;
    }

    if (CurInput == "" && currentValue != null)
    {
        pendingOperation = Op;
        return;
    }

    pendingOperation = Op;
    isNewInput = true;

    UpdateDisplay();
}

```

Рисунок 6 – OperationLogic

3. Equals_Click — выполняет вычисление, вызывая метод Calculate из библиотеки.

```

Ссылка: 1
private void Equals_Click(object sender, RoutedEventArgs e)
{
    if (pendingOperation != null && !isNewInput && CurInput != "")
    {
        double currentNumber = double.Parse(CurInput, CultureInfo.InvariantCulture);

        if (currentValue != null)
        {
            double result = Calc.Calculate((double)currentValue, currentNumber, pendingOperation);
            currentValue = result;
            CurInput = result.ToString(CultureInfo.InvariantCulture);
            pendingOperation = null;
            isNewInput = true;
            ResultWindow.Text = CurInput;
        }
    }
}

```

Рисунок 7 – Equals_Click

Для корректного парсинга чисел с плавающей точкой используется `CultureInfo.InvariantCulture`.

Модульное тестирование

Для тестирования библиотеки классов был создан отдельный проект с модульными тестами на базе фреймворка MSTest.

Создание тестового проекта:

1. Добавлен новый проект типа «MSTest Test Project».
2. Добавлена ссылка на проект CalculatorLibrary.
3. Написаны тесты для проверки всех операций.

```
using Microsoft.VisualStudio.TestTools.UnitTesting;
using System;
using CalculatorLibrary;

namespace CalculatorTests
{
    [TestClass]
    Ссылка: 0
    public class CalculatorTests
    {
        private Calculator Calc = new Calculator();

        [TestMethod]
        Ссылка: 0
        public void Add_TwoPositiveNumbers()
        {
            // Arrange
            double a = 5;
            double b = 3;
            double expected = 8;

            // Act
            double result = Calc.Calculate(a,b,"+");

            // Assert
            Assert.AreEqual(expected, result);
        }

        [TestMethod]
        Ссылка: 0
        public void Add_PositiveAndNegativeNumbers()
        {
            // Arrange
            double a = 5;
            double b = -3;
            double expected = 2;
        }
    }
}
```

Рисунок 8 – проект с модульными тестами

Все тесты успешно проходят, что подтверждает корректность реализации математических операций и правильную обработку исключительных ситуаций.

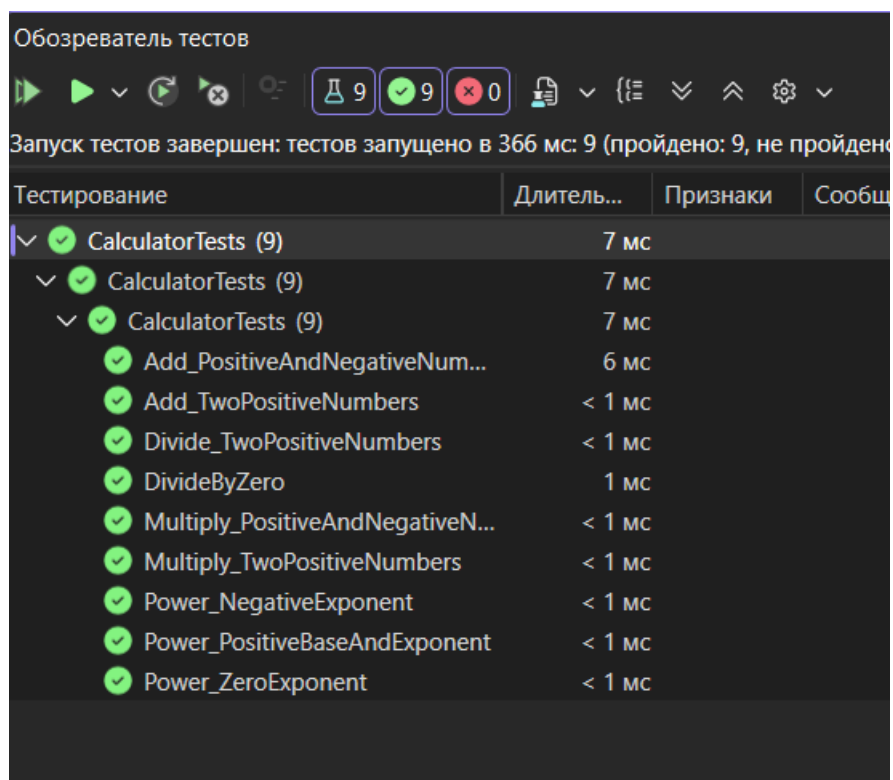


Рисунок 9 – результаты тестов

Заключение

В ходе выполнения практической работы был разработан программный продукт «Калькулятор», полностью удовлетворяющий поставленным требованиям.

Основные достигнутые результаты:

1. Создана модульная архитектура с разделением логики и интерфейса.
2. Реализована библиотека классов CalculatorLibrary с методами для выполнения пяти арифметических операций.
3. Разработан удобный пользовательский интерфейс с использованием WPF.

4. Написаны и успешно выполнены модульные тесты для всех методов библиотеки.

5. Обеспечена корректная обработка ошибок.

Полученные навыки:

1. Работа с технологией WPF для создания графических приложений
2. Проектирование и реализация библиотек классов.
3. Написание модульных тестов с использованием MSTest.
4. Интеграция различных компонентов в единое решение.

Все задачи, поставленные в рамках практической работы, выполнены в полном объёме. Приложение готово к использованию и соответствует требованиям технического задания.